



CURSO DE FIRMWARE

C para Sistemas Embebidos de 32 bits

"Fundamentos Sólidos para Firmware Seguro y Eficiente"

Proyecto

José Tepetla
Ing. de Software Senior



2026



Módulo de Control de Seguridad



Sistema de Seguridad para Electrodoméstico



Sensor Puerta



Sensor Temperatura



Control Motor

MCU 32-bit con UL 60730 Compliance



OBJETIVO: Construir un sistema de seguridad modular que integre todos los conceptos del curso

FASE 1: Variables y Máquina de Estados

TAREA:

"Crear las variables de un sensor de puerta usando tipos explícitos y un bucle principal con máquina de estados condicional."

```
//  TAREAS A COMPLETAR
// 1. Definir variables con stdint.h:
//   - puerta_cerrada (bool)
//   - temperatura (uint16_t)
//   - estado_sistema (enum con REPOS0, TRABAJO, FALLA)

// 2. Bucle principal con switch-case
while (1) {
    switch (estado_sistema) {
        case REPOS0:    // Verificar puerta
        case TRABAJO:   // Operación normal
        case FALLA:     // Manejo de error
        default:        // Estado seguro
    }
}
```

 [Compilador: onlinegdb.com](https://onlinegdb.com)

 **Tiempo: 20 minutos**

FASE 2: Modularización y volatile

TAREA:

"Modularizar el código en funciones usando punteros y proteger las banderas globales críticas con el modificador `volatile`."

```
//  TAREAS A COMPLETAR
// 1. Crear funciones:
//   - bool sistema_verificar_seguridad(void)
//   - void sistema_manejar_falla(CodigoFalla_t codigo)
//   - void sistema_actualizar_estado(void)

// 2. Usar punteros para modificar variables externas
void sistema_actualizar_contador(uint32_t* contador) {
    (*contador)++;
}

// 3. Banderas críticas con volatile
volatile bool bandera_interrupcion = false;
```

 [Compilador: onlinegdb.com](https://onlinegdb.com)

 Tiempo: 25 minutos

FASE 3: Estructura Optimizada y Unión para Diagnóstico

TAREA:

"Empaquetar todo el estado del sistema en una estructura optimizada sin padding, lista para ser leída byte a byte a través de una unión para fines de diagnóstico de fallas."

```
//  TAREAS A COMPLETAR
// 1. Crear struct SistemaState_t optimizada
//    (ordenar por tamaño para eliminar padding)

// 2. Crear union para diagnóstico
typedef union {
    SistemaState_t campos;
    uint8_t bytes[sizeof(SistemaState_t)];
} DiagnosticoUnion_t;

// 3. Función para enviar diagnóstico por UART
void sistema_enviar_diagnostico(DiagnosticoUnion_t* diag) {
    for (int i = 0; i < sizeof(DiagnosticoUnion_t); i++) {
        uart_transmitir(diag->bytes[i]);
    }
}
```

 [Compilador: onlinegdb.com](https://onlinegdb.com)

 **Tiempo: 20 minutos**

¡Felicidades! Curso Completado

✓ Lo que Aprendiste

- ✓ Palabras reservadas
- ✓ Tipos de datos portables (`stdint.h`)
- ✓ Operadores lógicos vs bits
- ✓ Máquinas de estado con `switch-case`
- ✓ Funciones y `enum` para parámetros legibles
- ✓ Punteros y paso por referencia
- ✓ `static` y `volatile` para firmware robusto
- ✓ `struct` y `union` para datos y protocolos
- ✓ Optimización de padding y macros seguras

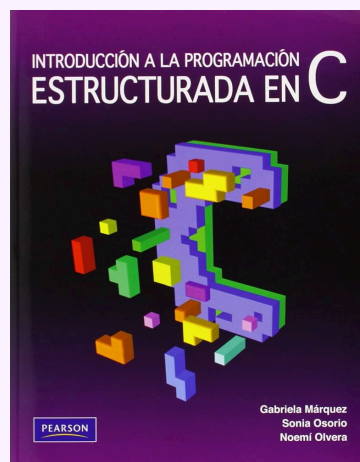
Recursos Adicionales

- **Documentación:** UL 60730 Standard
- **Documentación:** MISRA Standard
- Libro: ["Making Embedded Systems" - Elecia White](#)
- **Herramientas:** Compilador Online, IDE, Debugger

Próximos Pasos

- Implementar en hardware real (STM32, PIC, etc.)
- Agregar Watchdog Timer
- Implementar comunicación UART para diagnóstico

Bibliografía Recomendada



Preguntas y Respuestas - Q&A

RECAPITULANDO

 Tipos de Datos Operadores Bucles y Switch Funciones Punteros static/volatile struct/union Padding/Macrosgithub.com/AngelTepetla/Curso-Basico-Cjangel.tepetla@gmail.com

¡GRACIAS POR SU PARTICIPACIÓN!

Curso de Firmware - C para Sistemas Embebidos de 32 bits

[Volver al inicio](#)

J. Tepetla · 2026

Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)